

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1

По дисциплине «Модели решения задач в интеллектуальных системах»

Тема: «Реализация многослойного персептрона для решения задачи «исключающее ИЛИ»

Выполнил:

Студент 4 курса

Группы ИИ-26

Пасевич К.Ю.

Проверил:

Никитин С.К.

Брест 2026

Цель работы: реализовать многослойный персептрон (MLP) архитектуры 2-2-1 с сигмоидной функцией активации для решения задачи типа «исключающее ИЛИ» (XOR), в которой входные и выходные значения представлены не бинарными 0 и 1, а произвольными вещественными (целыми) числами из диапазона от -10 до 10 , соответствующими логическим классам 0 и 1. Провести сравнение эффективности обучения и качества решения задачи многослойным персептроном в сопоставлении с однослойным персептроном, показав принципиальную линейную неразделимость задачи XOR.

Ход работы

Задачи лабораторной работы

1. Реализовать многослойный персептрон архитектуры 2-2-1 (2 входных нейрона, 2 нейрона скрытого слоя, 1 выходной нейрон) с сигмоидной функцией активации на всех слоях.
2. Реализовать алгоритм обучения сети методом обратного распространения ошибки (backpropagation) в онлайн-режиме (обновление весов после каждого примера).
3. Обучить сеть на полной таблице истинности (4 примера) своего варианта до достижения заданного критерия остановки по суммарной ошибке $E_s \leq E_e$.
4. Реализовать и обучить однослойный персептрон (без скрытого слоя, один сигмоидный нейрон) на тех же данных и зафиксировать результат его обучения.
5. Сравнить многослойный и однослойный персептроны по следующим критериям:
 - количество эпох до сходимости (либо факт недостижения критерия остановки за заданное максимальное число эпох),
 - итоговая суммарная ошибка, точность (ассигасу) воспроизведения всех 4 примеров таблицы истинности.
6. Реализовать режим функционирования обученной сети: пользователь вводит пару чисел (A, B) из диапазона $[-10; 10]$ (не обязательно совпадающих с c_0 и c_1), сеть выдаёт выходное значение $\hat{y}$ и указывает, к какому классу (c_0 или c_1) оно ближе.

Вариант 9 ($c_0 = 3, c_1 = -8$)

A	B	$A \oplus B$
3	3	3
3	-8	-8
-8	3	-8
-8	-8	3

Код программы:

```
import numpy as np

c0, c1 = 3, -8

X = np.array([
    [c0, c0],
    [c0, c1],
    [c1, c0],
    [c1, c1]
], dtype=float)

Y_raw = np.array([
    [c0],
    [c1],
    [c1],
    [c0]
```

```

        [c1],
        [c0]
    ], dtype=float)

def normalize(x, xmin, xmax):
    return (x - xmin) / (xmax - xmin)

def denormalize(y_norm, xmin, xmax):
    return xmin + y_norm * (xmax - xmin)

Xn = normalize(X, c0, c1)
Y = normalize(Y_raw, c0, c1)

def sigmoid(S):
    return 1.0 / (1.0 + np.exp(-S))

def dsigmoid(y):
    return y * (1.0 - y)

class MLP221:
    def __init__(self, lr=0.5, seed=7):
        rng = np.random.RandomState(seed)
        self.W1 = rng.uniform(-0.1, 0.1, (2, 2))
        self.T1 = rng.uniform(-0.1, 0.1, (1, 2))
        self.W2 = rng.uniform(-0.1, 0.1, (2, 1))
        self.T2 = rng.uniform(-0.1, 0.1, (1, 1))
        self.lr = lr

    def forward(self, x):
        x = x.reshape(1, -1)
        y1 = sigmoid(x @ self.W1 - self.T1)
        y2 = sigmoid(y1 @ self.W2 - self.T2)
        return y1, y2

    def train_step(self, x, e):
        x, e = x.reshape(1, -1), e.reshape(1, -1)
        y1, y2 = self.forward(x)
        delta2 = (y2 - e) * dsigmoid(y2)
        delta1 = (delta2 @ self.W2.T) * dsigmoid(y1)

        self.W2 -= self.lr * (y1.T @ delta2)
        self.T2 += self.lr * delta2
        self.W1 -= self.lr * (x.T @ delta1)
        self.T1 += self.lr * delta1

        return 0.5 * np.sum((y2 - e) ** 2)

    def predict(self, x):
        return self.forward(x)[1][0, 0]

    def fit(self, X, Y, Ee=0.01, max_epochs=20000):
        for epoch in range(1, max_epochs + 1):
            idx = np.random.permutation(len(X))
            Es = sum(self.train_step(X[i], Y[i]) for i in idx)
            if Es <= Ee:
                return epoch, Es
        return max_epochs, Es

class SingleLayerPerceptron:
    def __init__(self, lr=0.5, seed=7):
        rng = np.random.RandomState(seed)

```

```

        self.W = rng.uniform(-0.1, 0.1, (2, 1))
        self.T = rng.uniform(-0.1, 0.1, (1, 1))
        self.lr = lr

    def forward(self, x):
        return sigmoid(x.reshape(1, -1) @ self.W - self.T)

    def train_step(self, x, e):
        x, e = x.reshape(1, -1), e.reshape(1, -1)
        y = self.forward(x)
        delta = (y - e) * dsigmoid(y)
        self.W -= self.lr * (x.T @ delta)
        self.T += self.lr * delta
        return 0.5 * np.sum((y - e) ** 2)

    def predict(self, x):
        return self.forward(x)[0, 0]

    def fit(self, X, Y, Ee=0.01, max_epochs=20000, seed=42):
        rng = np.random.RandomState(seed)
        for epoch in range(1, max_epochs + 1):
            idx = rng.permutation(len(X))
            Es = sum(self.train_step(X[i], Y[i]) for i in idx)
            if Es <= Ee:
                return epoch, Es
        return max_epochs, Es

def accuracy(model, Xn, Y, thr=0.5):
    correct = 0
    for x, y in zip(Xn, Y):
        pred_class = int(model.predict(x) >= thr)
        true_class = int(y[0] >= thr)
        correct += int(pred_class == true_class)
    return correct / len(Xn)

def run_inference(model, A, B, c0, c1, name="Model"):
    x = normalize(np.array([A, B], dtype=float), c0, c1)
    y_norm = model.predict(x)
    y_hat = denormalize(y_norm, c0, c1)
    nearest = c0 if y_norm < 0.5 else c1
    print(f"[{name}] A={A}, B={B} -> y_hat={y_hat:.3f} -> closest class {nearest}")
    return y_hat, nearest

if __name__ == "__main__":
    mlp = MLP221()
    mlp_epochs, mlp_Es = mlp.fit(Xn, Y)

    slp = SingleLayerPerceptron()
    slp_epochs, slp_Es = slp.fit(Xn, Y)

    print("MLP 2-2-1:", mlp_epochs, "epochs,", round(mlp_Es, 5), "error,", "accuracy",
          accuracy(mlp, Xn, Y))
    print("SLP:", slp_epochs, "epochs,", round(slp_Es, 5), "error,", "accuracy =",
          accuracy(slp, Xn, Y))

    while True:
        raw = input("Enter A B (empty line to exit): ").strip()
        if not raw:
            break
        A, B = map(float, raw.split())

```

```
run_inference(mlp, A, B, c0, c1, "MLP")
run_inference(slp, A, B, c0, c1, "SLP")
```

Результат программы:

MLP 2-2-1: 3043 epochs, 0.00999 error, accuracy = 1.0

SLP: 20000 epochs, 0.53145 error, accuracy = 0.75

Enter A B (empty line to exit): 3 -8

[MLP] A=3.0, B=-8.0 -> y_hat=-7.267 -> closest class -8

[SLP] A=3.0, B=-8.0 -> y_hat=-2.542 -> closest class -8

Enter A B (empty line to exit): -8 -8

[MLP] A=-8.0, B=-8.0 -> y_hat=2.029 -> closest class 3

[SLP] A=-8.0, B=-8.0 -> y_hat=-2.461 -> closest class 3

Enter A B (empty line to exit): -8 3

[MLP] A=-8.0, B=3.0 -> y_hat=-7.267 -> closest class -8

[SLP] A=-8.0, B=3.0 -> y_hat=-2.505 -> closest class -8

Enter A B (empty line to exit): 3 3

[MLP] A=3.0, B=3.0 -> y_hat=2.384 -> closest class 3

[SLP] A=3.0, B=3.0 -> y_hat=-2.586 -> closest class -8

Enter A B (empty line to exit):

Вывод: выполнив данную лабораторную работу, я разработала многослойный персептрон (MLP) с архитектурой 2-2-1 и сигмоидной функцией активации, адаптировав его для решения задачи XOR. Особенностью реализации стало использование вместо классических бинарных нулей и единиц произвольных вещественных (целых) чисел из интервала от -10 до 10 , кодирующих логические классы. В ходе сравнительного анализа эффективности обучения и точности аппроксимации многослойной сети в сопоставлении с однослойным персептроном я наглядно продемонстрировала математическую невозможность решения задачи XOR одной разделяющей гиперплоскостью в силу её принципиальной линейной неразделимости.